

系统开发工具基础

第四周实验报告

乔海轩—25090012035

2026 年 9 月 14 日

目录

1 总览	2
2 第 13 题建立可执行的本地质量门禁	3
3 第 14 题让 Make 只重建真正受影响的产物	5
4 第 15 题把本地 API 数据转换为可读报告	6
5 第 16 题修复并交付一个陌生的小型工具仓库	8

1 总览

题目编号	描述	完成情况
13	建立可执行的本地质量门禁	已完成
14	让 Make 只重建真正受影响的产物	已完成
15	把本地 API 数据转换为可读报告	已完成
16	修复并交付一个陌生的小型工具仓库	已完成

Github 仓库地址: <https://github.com/Ptilotsis-o/system-dev-tools/>
Commit 记录如图 1 和 2 所示。

```
* 894ec93 feat(q13): add check.sh
* 86c893c fix(q13): Packaged cli.py. Now it's actually all passed
* 6f2e0b3 feat(q13): reformatted and all tests passed
* f512324 fix(q13): fix .toml. Run ruff format
* aede7a5 feat(q13): improve tests file
* 0669372 feat(q13): improve .toml file
```

图 1: Commit 记录截图

```
* a1acacd (HEAD -> master, origin/master, origin/HEAD) chore(q16): remove tmp dir
* 17134de Add 'week4/q16/' from commit '666ddb3e81e0a62b3f1af9e55d5b345d61c6525b'
\
* 666ddb3 build(q16): add Makefile with check, build, and clean targets
* 01f2e4a fix(q16): use actual --name value in greeting output
* a18558c feat(q16): error occurred
* 99506f7 chore: import greetlab from q13
* f930f9f chore: remove q16 before subtree merge
* 7bdcdb5 feat(q16): complete cp
* 7cbbced feat(q15): generate .md file
* 09b31ed feat(q15): write api_report.sh
* 91c3e98 feat(q15): file prepared
* 0999274 feat(q14): make clean
* 78ffc53 feat(q14): make done
* 179e2de fix(q14): fix Makefile
* 8653de2 feat(q14): build Makefile
* 4770ed0 feat(q14): files prepared
```

图 2: Commit 记录截图

2 第 13 题建立可执行的本地质量门禁

题目要求

- 复制 q10 为 q13; 在 pyproject.toml 中加入 ruff 和 pytest 的最小配置。
- 补充一个正常姓名测试和一个空白姓名测试, 每个测试包含有效断言。
- 运行 ruff format、ruff check 和 pytest, 修复全部问题, 不得全局忽略规则。
- 编写 check.sh, 依次执行 ruff format -check、ruff check 和 pytest; 运行并保证返回 0。

操作步骤与命令

```
cp -r ../week3/q10 q13
cd q13
vim pyproject.toml
vim tests/test_cli.py
ruff format .
ruff check .
pytest -v
vim check.sh
chmod +x check.sh
./check.sh
echo $?
```

结果验证

- pyproject.toml 中加入 ruff 最小配置 (target-version、line-length、lint 的 select 列表, ignore 为空) 与 pytest 最小配置 (testpaths 为 tests);
- test_normal_name 与 test_blank_name_exits 各自包含有效断言, 均通过;
- ruff format、ruff check 无任何报错, 未使用全局忽略规则;

- check.sh 依次执行 `ruff format --check`、`ruff check` 与 `pytest`, 运行结束打印 `All checks passed`, `echo $?` 输出 0。

```
(env) qiao-haixuan@qiao-haixuan-Virtual-Platform:~/system-dev-tools/week4/q13$ ruff format
4 files left unchanged
(env) qiao-haixuan@qiao-haixuan-Virtual-Platform:~/system-dev-tools/week4/q13$ ruff check
E402 Module level import not at top of file
--> tests/test_cli.py:9:1
|
|   |
7 | sys.path.insert(0, str(src_path))
8 |
9 | from greetlab.cli import main
| ~~~~~
help: Move module level imports to top of file

Found 1 error.
(env) qiao-haixuan@qiao-haixuan-Virtual-Platform:~/system-dev-tools/week4/q13$ pytest -v
===== test session starts =====
platform linux -- Python 3.14.4, pytest-9.1.1, pluggy-1.6.0 -- /home/qiao-haixuan/system-dev-tools/week3/q09/env/bin/python3.14
cachedir: .pytest_cache
rootdir: /home/qiao-haixuan/system-dev-tools/week4/q13
configfile: pyproject.toml
testpaths: tests
collected 2 items

tests/test_cli.py::test_normal_name PASSED [ 50%]
tests/test_cli.py::test_blank_name_exits PASSED [100%]

===== 2 passed in 0.01s =====
```

图 3: ruff format , ruff check 与 pytest 输出截图

```
(env) qiao-haixuan@qiao-haixuan-Virtual-Platform:~/system-dev-tools/week4/q13$ ./check.sh
=== Running Ruff Format (Check) ===
4 files already formatted
=== Running Ruff Lint ===
All checks passed!
=== Running Pytest ===
===== test session starts =====
platform linux -- Python 3.14.4, pytest-9.1.1, pluggy-1.6.0 -- /home/qiao-haixuan/system-dev-tools/week3/q09/env/bin/python3.14
cachedir: .pytest_cache
rootdir: /home/qiao-haixuan/system-dev-tools/week4/q13
configfile: pyproject.toml
testpaths: tests
collected 2 items

tests/test_cli.py::test_normal_name PASSED [ 50%]
tests/test_cli.py::test_blank_name_exits PASSED [100%]

===== 2 passed in 0.01s =====

All checks passed
(env) qiao-haixuan@qiao-haixuan-Virtual-Platform:~/system-dev-tools/week4/q13$ echo $?
0
```

图 4: check.sh 运行及退出码截图

3 第 14 题让 Make 只重建真正受影响的产物

题目要求

- 创建给定的 data.csv、stats.py、build_report.py 和 report.md。
- 编写 Makefile，包含 all、stats.txt、report.txt 和 clean；准确列出数据文件、脚本和中间产物依赖，并把 all、clean 声明为.PHONY。
- 验证首次 make 生成两个产物；第二次无改动时不执行配方。
- touch data.csv 后再次 make，确认 stats.txt 和 report.txt 重新生成；clean 只删除生成物。

操作步骤与命令

```
vim data.csv stats.py build_report.py report.md
vim Makefile
make
make
touch data.csv
make
make clean
ls
```

结果验证

- 依赖关系准确：stats.txt 依赖 data.csv 与 stats.py，report.txt 依赖 report.md 与 stats.txt；all、clean 已声明为.PHONY；
- 首次 make 由 data.csv 计算得到 stats.txt（内容为 10），再生成 report.txt；
- 第二次 make 未执行任何配方，只提示'stats.txt' is up to date；
- touch data.csv 后再次 make，stats.txt 与 report.txt 均重新生成，时间戳更新；
- make clean 只删除 stats.txt 与 report.txt，源文件与 Makefile 保留。

```
(venv) qiao-haixuan@qiao-haixuan-VMware-Virtual-Platform:~/system-dev-tools/week4/q14$ make
python stats.py
python build_report.py
(venv) qiao-haixuan@qiao-haixuan-VMware-Virtual-Platform:~/system-dev-tools/week4/q14$ make
make: 对“all”无需做任何事。
```

图 5: 首次 make 生成两个产物截图和第二次 make 不执行配方截图

```
(venv) qiao-haixuan@qiao-haixuan-VMware-Virtual-Platform:~/system-dev-tools/week4/q14$ touch data.csv
(venv) qiao-haixuan@qiao-haixuan-VMware-Virtual-Platform:~/system-dev-tools/week4/q14$ make
python stats.py
python build_report.py
```

图 6: touch 后重新生成截图

4 第 15 题把本地 API 数据转换为可读报告

题目要求

- 把给定 JSON 保存为 packages.json，并在 q15 目录运行 `python -m http.server 8000`。
- 使用 `curl -fsS` 获取 `http://127.0.0.1:8000/packages.json`。
- 使用 `jq` 筛选 status 为 active 且 downloads 不少于 100 的记录，并按 downloads 降序、name 升序排列。
- 编写 `api_report.sh`，把结果生成 `summary.md`，包含标题和 name、version、downloads 三列表格。

操作步骤与命令

```
vim packages.json
python -m http.server 8000 &
curl -fsS http://127.0.0.1:8000/packages.json
vim api_report.sh
chmod +x api_report.sh
./api_report.sh
cat summary.md
```

结果验证

- packages.json 保存正确, `python -m http.server 8000` 在 q15 目录提供该文件;
- `curl -fsS` 成功取回 JSON 内容, 未依赖浏览器或手工复制;
- jq 筛选出 alpha、gamma、delta 三条记录 (beta 为 inactive、epsilon 下载量不足 100 被排除), 排序为 downloads 降序、同值按 name 升序: delta (450)、gamma (450)、alpha (120);
- summary.md 包含标题 `# Active Packages` 与 name、version、downloads 三列表格, 共三行数据, 与 `api_report.sh` 的输出一致。

```
(venv) qiao-haixuan@qiao-haixuan-Virtual-Platform:~/system-dev-tools/week4/q15$ python -m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
```

图 7: http.server 截图

```
qiao-haixuan@qiao-haixuan-Virtual-Platform:~/system-dev-tools/week4/q15$ curl -fsS http://127.0.0.1:8000/packages.json
[
  {"name": "alpha", "status": "active", "downloads": 120, "version": "1.2.0"},
  {"name": "beta", "status": "inactive", "downloads": 900, "version": "2.0.0"},
  {"name": "gamma", "status": "active", "downloads": 450, "version": "1.5.1"},
  {"name": "delta", "status": "active", "downloads": 450, "version": "0.9.0"},
  {"name": "epsilon", "status": "active", "downloads": 80, "version": "3.1.0"}
]
```

图 8: curl 捕获截图

```
qiao-haixuan@qiao-haixuan-Virtual-Platform:~/system-dev-tools/week4/q15$ cat summary.md
# Active Packages

| name | version | downloads |
| --- | --- | --- |
| delta | 0.9.0 | 450 |
| gamma | 1.5.1 | 450 |
| alpha | 1.2.0 | 120 |
```

图 9: summary.md 内容截图

5 第 16 题修复并交付一个陌生的小型工具仓库

题目要求

- 复制 q13 为 q16 并初始化/继续使用 Git。
- 把问候语实现临时改成始终返回字面量 Hello, name!, 运行 check.sh 确认测试失败; 随后定位并修复。
- 编写最小 Makefile, 包含 check、build 和 clean; check 调用 check.sh、build 生成 wheel。
- 运行 make check 和 make build, 计算 wheel 的 SHA-256, 并把最终改动提交为一个内容聚焦的 Git 提交。

操作步骤与命令

```
cp -r q13 q16
cd q16
git init
git add -A
git commit -m "chore: import greetlab from q13"
vim src/greetlab/cli.py # 临时改动
./check.sh
vim src/greetlab/cli.py # 修复
./check.sh
vim Makefile
make check
make build
sha256sum dist/greetlab_25090012035-0.1.0-py3-none-any.whl
git add -A
git commit -m "..."
```

结果验证

- q16 由 q13 复制而来并纳入 Git 管理;

- 临时改动后 check.sh 返回非 0，test_normal_name 断言失败，实际输出为 Hello, name! 而非 Hello, Alice!;
- 修复后 make check 返回 0，ruff format --check、ruff check 与 pytest 全部通过;
- make build 生成 greetlab_25090012035-0.1.0-py3-none-any.whl，并计算出该 wheel 的 SHA-256;
- 最终改动以单个内容聚焦的提交入库 (fix(cli): restore blank-name validation)。

```

testpaths: tests
collected 2 items

tests/test_cli.py::test_normal_name FAILED [ 50%]
tests/test_cli.py::test_blank_name_exits PASSED [100%]

===== FAILURES =====
test_normal_name

capsys = <_pytest.capture.CaptureFixture object at 0x7498eb3ba310>

    def test_normal_name(capsys):
        sys.argv = ["sdt-greet", "--name", "Alice"]
        main()
        captured = capsys.readouterr()
        assert captured.out.strip() == "Hello, Alice!"
> AssertionError: assert 'Hello, name!' == 'Hello, Alice!'
E
E       - Hello, Alice!
E       + Hello, name!
E

tests/test_cli.py:12: AssertionError
===== short test summary info =====
FAILED tests/test_cli.py::test_normal_name - AssertionError: assert 'Hello, name!' == 'Hello, Alice!'
===== 1 failed, 1 passed in 0.03s =====

```

图 10: 临时改动后 check.sh 测试失败截图

```

(env) qiao-haixuan@qiao-haixuan-Virtual-Platform:~/system-dev-tools/week4/q16$ make check
./check.sh
=== Running Ruff Format (Check) ===
4 files already formatted
=== Running Ruff lint ===
All checks passed!
=== Running Pytest ===
===== test session starts =====
platform linux -- Python 3.14.4, pytest-9.1.1, pluggy-1.6.0 -- /home/qiao-haixuan/system-dev-tools/week3/q09/env/bin/python3.14
cachedir: .pytest_cache
rootdir: /home/qiao-haixuan/system-dev-tools/week4/q16
configfile: pyproject.toml
testpaths: tests
collected 2 items

tests/test_cli.py::test_normal_name PASSED [ 50%]
tests/test_cli.py::test_blank_name_exits PASSED [100%]

===== 2 passed in 0.01s =====
All checks passed

```

图 11: make check 输出截图

```
(env) qiao-haixuan@qiao-haixuan-Virtual-Platform:~/system-dev-tools/week4/q16$ sha256sum dist/greetlab_25090012035-0.1.0-py3-none-any.whl
21940c53c6c862c7976e62ccdf92e169fb3cfe7f4d4383a6907283a6a170e2f6  dist/greetlab_25090012035-0.1.0-py3-none-any.whl
```

图 12: wheel 的 SHA-256 计算结果截图

```
* 666ddb3 build(q16): add Makefile with check, build, and clean targets
* 01f2e4a fix(q16): use actual --name value in greeting output
* a18558c feat(q16): error occurred
* 99506f7 chore: import greetlab from q13
```

图 13: 最终 Git 提交记录截图